
HematoScan

ML Model Architecture & Prediction Pipeline

Blood Cancer Detection System - Technical Documentation

Table of Contents

1. Overview
2. Models Used (No CNN)
3. Blood Test (CBC) Prediction
4. Image Analysis Prediction
5. End-to-End Prediction Flow
6. Training Data & Caveats
7. Fallback Mechanism
8. Getting Started with Real Data

1. Overview

HematoScan uses machine learning to predict blood cancer risk from two types of input: Complete Blood Count (CBC) parameters and blood smear microscopy images. Both models are classic scikit-learn ensemble classifiers, not deep neural networks.

The system is built with a two-tier prediction architecture: trained ML models as the primary predictor, with a rule-based fallback when models are unavailable. This ensures the application remains functional even during development.

2. Models Used (No CNN)

Neither model uses a Convolutional Neural Network (CNN) or any deep learning. Both are tree-based ensemble methods from scikit-learn, chosen for their interpretability, fast inference, and good performance on tabular datasets.

Model	Algorithm	Type	Parameters
Blood Test (CBC)	RandomForestClassifier	Ensemble (Bagging)	200 trees, depth 12
Image Analysis	GradientBoostingClassifier	Ensemble (Boosting)	150 stages, lr=0.1

Neither TensorFlow, PyTorch, nor any deep learning framework is used in the project. Both models are serialized using joblib (pickle protocol 5) and loaded into scikit-learn Pipelines at runtime.

3. Blood Test (CBC) Prediction

Input Features (10 CBC Parameters)

- WBC (White Blood Cells) - $\times 10^3/\mu\text{L}$
- RBC (Red Blood Cells) - $\times 10^6/\mu\text{L}$
- Hemoglobin - g/dL
- Platelets - $\times 10^3/\mu\text{L}$
- Neutrophils - %
- Lymphocytes - %
- Monocytes - %
- Eosinophils - %
- Basophils - %
- Blast Cells - %

Pipeline Architecture

The blood test model uses a scikit-learn Pipeline with two steps:

```
Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", RandomForestClassifier(
        n_estimators=200,
        max_depth=12,
        min_samples_split=5,
        min_samples_leaf=2,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1,
    )),
])
```

Step 1: StandardScaler

All input values are normalized by subtracting the mean and dividing by the standard deviation for each feature. This prevents features with larger numerical ranges (e.g., platelets: 150-450) from dominating features with smaller ranges (e.g., basophils: 0-1). The scaler is fitted on the training data.

Step 2: RandomForestClassifier

An ensemble of 200 decision trees. Each tree is trained on a random bootstrap sample of the data with random feature subsets. At prediction time, each tree votes for one of three classes (Normal, Benign, Malignant). The final prediction is the majority vote. The confidence score is the fraction of trees that voted for the winning class.

Confidence Calculation

For a Random Forest with 200 trees: if 180 predict 'Normal', 15 predict 'Benign', and 5 predict 'Malignant', the result is 'Normal' with confidence $180/200 = 0.90$ (90%). Derived from `model.predict_proba()`.

4. Image Analysis Prediction

The image model does NOT process raw pixels or use CNNs. Instead, it uses OpenCV to extract 10 quantitative features from the blood smear image, then feeds those numerical features into a GradientBoostingClassifier.

Feature Extraction Pipeline

Features extracted using OpenCV image processing:

- Cell Count - Cells detected via contour detection on Otsu-thresholded image
- Cell Size Mean - Average area of detected cell contours
- Cell Size Std - Standard deviation of contour areas
- Nucleus Ratio Mean - Dark pixel (nucleus) area / total cell area
- Nucleus Ratio Std - Constant estimate (0.05)
- Blue Intensity - Mean blue channel (normalized 0-1)
- Red Intensity - Mean red channel (normalized 0-1)
- Texture Contrast - Standard deviation of grayscale (0-1)
- Texture Homogeneity - 1.0 minus texture contrast
- Blast-like Cells % - Cells with nucleus ratio > 50%

OpenCV Processing Steps

1. Read image with cv2.imread(), convert to grayscale
2. Apply GaussianBlur to reduce noise
3. Otsu's thresholding for binary segmentation (cell vs background)
4. Find contours with cv2.findContours()
5. Filter contours by minimum area (20 px)
6. For each contour: calculate area, bounding box, nucleus ratio
7. Compute aggregate statistics across all contours

Gradient Boosting Classifier

```
Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", GradientBoostingClassifier(
        n_estimators=150,
        max_depth=8,
        min_samples_split=5,
        learning_rate=0.1,
        random_state=42,
    )),
])
```

Unlike Random Forest (parallel trees), Gradient Boosting builds trees sequentially. Each new tree corrects the errors of the previous ensemble. With 150 trees and learning rate 0.1, the model gradually improves. This often yields higher

accuracy on tabular data but is more sensitive to hyperparameters.

5. End-to-End Prediction Flow

```

User submits data (CBC form or blood smear image)
      |
      v
prediction.py
      |
      +---> Are .pkl model files available?
      |     |
      |     YES: model_service.py
      |           +---> Load model with joblib
      |           +---> Prepare feature vector
      |           +---> model.predict(features)
      |           +---> model.predict_proba(features)
      |           +---> Return (label, confidence)
      |
      |     NO: fallback (prediction.py)
      |           +---> Rule-based scoring
      |           +---> Map risk score to class
      |           +---> Return (label, confidence)
      |
      v
Returns JSON:
{ prediction: 'Normal'|'Benign'|'Malignant',
  confidence: 0.00-0.99 }

```

The prediction service (prediction.py) checks if trained .pkl model files exist via `model_service.models_available()`. If they do, it delegates to the ML model. If not, it uses a rule-based heuristic. This dual-path architecture keeps the frontend functional during development before models are trained.

6. Training Data & Caveats

Synthetic Data Generation

Both models were trained on synthetically generated data, not real patient data. The data generation module creates medically-plausible samples using predefined normal ranges and deviation patterns for each class.

CBC Class Patterns (Feature Means)

Parameter	Normal	Benign	Malignant	Unit
WBC	7.5	11.0	18.0	$\times 10^3/\mu\text{L}$
RBC	5.2	4.5	3.8	$\times 10^6/\mu\text{L}$
Hemoglobin	14.5	12.0	9.5	g/dL
Platelets	250	180	120	$\times 10^3/\mu\text{L}$
Blast Cells	0.5%	5.0%	18.0%	%

Training Configuration

- 800 samples per class (2,400 total per dataset)
- 80/20 train/test split with stratification
- 5-fold cross-validation
- Fixed random seed (42) for reproducibility
- Datasets exported to CSV in data/ directory

Important Caveat

Models learn patterns from simulated data, not real clinical data. While the synthetic data follows medically-plausible distributions (malignant = higher WBC, lower hemoglobin, more blast cells), the models have never seen real patient results or microscopy images. Suitable for demonstration only.

7. Fallback Mechanism

When the trained .pkl files are unavailable, a rule-based algorithm is used.

Blood Test Fallback Rules

Risk score (0.0-1.0) based on clinical heuristics:

- Blast cells > 20%: +0.8 risk
- Blast cells 10-20%: +0.5 risk
- Blast cells 5-10%: +0.3 risk
- WBC > 15 or < 3: +0.2 risk
- Hemoglobin < 10 g/dL: +0.15 risk
- Platelets < 100: +0.2 risk
- Neutrophils > 85% or < 20%: +0.1 risk
- Random noise (+/- 0.05) added

Risk mapped to prediction:

- Risk < 0.3 -> 'Normal' (confidence: 85-99%)
- Risk 0.3-0.6 -> 'Benign' (confidence: 75-94%)
- Risk > 0.6 -> 'Malignant' (confidence: 80-95%)

Image Fallback

Weighted random selection:

- 60% Normal
- 25% Benign
- 15% Malignant

Confidence randomly assigned within a plausible range per class.

8. Getting Started with Real Data

Train with Your Own CSV Data

```
# Blood test CSV columns:
wbc,rbc,hemoglobin,platelets,neutrophils,
lymphocytes,monocytes,eosinophils,basophils,
blast_cells,target

# Image feature CSV columns:
cell_count,cell_size_mean,cell_size_std,
nucleus_ratio_mean,nucleus_ratio_std,
blue_intensity,red_intensity,
texture_contrast,texture_homogeneity,
```



```
blast_like_cells_pct,target

# target: Normal, Benign, or Malignant
```

Training Commands

```
# Train from CSV files:
python -m app.ml.train \
  --csv-cbc ./data/cbc.csv \
  --csv-img ./data/image.csv

# Auto-detect latest CSV:
python -m app.ml.train

# Force synthetic:
python -m app.ml.train --generate

# Export datasets + train:
python -m app.ml.train --export

# Show dataset stats:
python -m app.ml.train --stats ./data/cbc.csv
```

Upgrade Path to Deep Learning

To add a CNN or deep learning model:

1. Add TensorFlow/PyTorch to requirements.txt
2. Create app/ml/train_cnn.py
3. Update model_service.py to load the new model
4. Update prediction.py routing

The clean separation of concerns means API routes and frontend require no changes.